UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 1

STUDENT: _____
GRUPA: _____

EXAMEN LA DISCIPLINA "PROGRAMARE AVANSATĂ PE OBIECTE ÎN JAVA" – SESIUNEA IUNIE 2026 –

NOTĂ:

- Datele de intrare se consideră corecte.
- Nu se vor trata excepţiile şi importurile.
- Punctaj: 5p. (10 x 0.5p.) + 2p. + 2p. + 1p. (din oficiu)

I. Pentru fiecare dintre cele 5 întrebări de mai jos, indicaţi variantele de răspuns pe care le consideraţi corecte:

1. Fie următorul program Java:

```
class Automobil {
    private String marca;
    public Automobil(String marca) { this.marca = marca; }
    public int hashCode() { return marca.length() / 2; }
    public boolean equals(Object obj) {
        return marca.charAt(0) != ((Automobil)obj).marca.charAt(0); }
}

public class Test {
    public static void main(String[] args) {
        HashSet<Automobil> ma = new HashSet<>();
        ma.add(new Automobil("Dacia")); ma.add(new Automobil("Bugatti"));
        ma.add(new Automobil("Ford"));  ma.add(new Automobil("Buick"));
        ma.add(new Automobil("Fiat"));  ma.add(new Automobil("Delage"));
        System.out.println(ma.size());
    }
}
```

După executarea programului, se va afişa valoarea:
a) 1 **b)** 2 **c)** 3 **d)** 4

2. Ştiind faptul că şirul de caractere `s` conţine doar paranteze rotunde, deschise sau închise, ce se va afişa pe ecran după executarea secvenţei de mai jos?

```
int n = 0;
int p = s.indexOf("()");
while (p != -1) {
    n++;
    s = s.substring(p + 2);
    p = s.indexOf("()");
}
System.out.println(n);
```

a) numărul de paranteze din prima grupare consecutivă de perechi `()` din şirul `s`
b) numărul perechi `()` care apar în şirul iniţial `s`, indiferent dacă sunt consecutive sau nu
c) numărul de paranteze rămase în şirul `s` după eliminarea tuturor perechilor corect închise
d) numărul de grupări de paranteze imbricate corect care conţin cel puţin o pereche de paranteze în interiorul altei perechi, de exemplu `(())`, `((()))` ş.a.m.d., din şirul `s`

3. Considerăm următorul program Java:

```
class A { public void print() { System.out.print("A "); } }
class B extends A { }
class C extends B { public void print() { System.out.print("B "); } }

public class Test {
    public static void main(String args[]) {
        A p = new B();
        B q = new B();
        A r = new C();
        p.print();
        q.print();
        r.print();
    }
}
```

După executarea programului, se va afişa:
a) `A B A` **b)** `A B B` **c)** `B B B` **d)** `A A B`

4. Într-o aplicaţie de gestiune urbană considerăm clasele `Cartier`, `Casa`, `Camera` şi `Proprietar`. Specificaţiile funcţionale ale aplicaţiei sunt următoarele:

- `Casa` este compusă din mai multe obiecte de tip `Camera`. Dacă o casă este demolată, atunci toate camerele sale dispar instantaneu.
- Fiecare `Casa` se află într-un `Cartier`. Dacă un `Cartier` este desfiinţat sau reorganizat administrativ, casele nu dispar, ci sunt mutate în mod dinamic în administrarea altui cartier.
- Fiecare `Casa` are un `Proprietar`. Dacă o casă este vândută sau demolată, obiectul `Proprietar` nu trebuie afectat, deoarece acesta poate deţine şi alte proprietăţi în aplicaţie sau poate rămâne în baza de date ca utilizator fără nicio casă.

Care dintre următoarele variante este o abordare corectă de modelare a acestor specificaţii din punct de vedere al programării orientate pe obiecte?

a) clasa `Casa` extinde clasa `Cartier`, clasa `Camera` extinde clasa `Casa`, iar între clasele `Proprietar` şi `Casa` avem o relaţie de agregare
b) între clasele `Cartier` şi `Casa` avem o relaţie de compoziţie, între clasele `Casa` şi `Camera` avem o relaţie de agregare, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
c) între clasele `Casa` şi `Camera` avem o relaţie de compoziţie, între clasele `Cartier` şi `Casa` avem o relaţie de agregare, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
d) se defineşte şi o clasă abstractă `Oraş` pe care o vor extinde direct toate cele 4 clase precizate anterior

5. Care dintre următoarele afirmaţii referitoare la o clasă abstractă sunt false?
a) constructorul unei clase abstracte C poate fi apelat direct folosind expresia `C.C()`
b) o clasă abstractă nu poate avea constructori, deoarece nu poate fi instanţiată direct cu operatorul `new`
c) toţi constructorii unei clase abstracte trebuie să fie obligatoriu `private`, pentru a împiedica instanţierea directă a clasei
d) o clasă abstractă poate avea constructori, iar aceştia vor fi apelaţi la crearea obiectelor claselor derivate, implicit sau explicit folosind cuvântul cheie `super`

6. Considerăm următorul program Java:

```
public class Test {
    public static void main(String args[]) throws FileNotFoundException {
        Scanner fin = new Scanner(new File("numere.txt"));
        int s = 0;

        while (fin.hasNext()) {
            try {
                int x = fin.nextInt();
                s = s + x;
            }
            catch (InputMismatchException e) {
                System.out.println("Număr incorect!");
                fin.next();
            }
        }
        System.out.println(s);
    }
}
```

Dacă în fişierul text *numere.txt* există cel puţin două numere scrise incorect (de exemplu: 2a17, a217, abc, 2-17 etc.), atunci programul:

a) va afişa suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina forţat din cauza unei excepţii
b) va afişa o singură dată mesajul `Număr incorect!` şi suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina normal
c) va afişa câte un mesaj `Număr incorect!` pentru fiecare număr din fişier scris incorect, apoi va afişa suma tuturor numerelor din fişier scrise corect, după care programul se va termina normal
d) va afişa doar mesajul `Număr incorect!`, după care programul se va termina forţat din cauza unei excepţii

7. Considerăm următorul cod Java:

```
class Fir extends Thread {
    int nivel;
    public Fir(int n) { nivel = n; }
    public void run() {
        System.out.print(nivel + " ");
        if (nivel < 3) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
        if (nivel < 1000) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Fir fir = new Fir(0);
        fir.start();
    }
}
```

Presupunând că toate firele create în programul de mai sus reuşesc să se execute complet, care dintre următoarele afirmaţii sunt adevărate?

a) valorile afişate vor apărea obligatoriu în ordine crescătoare, deoarece fiecare fir cu nivelul `n` creează fire doar cu nivelul `n + 1`

b) valoarea 999 va fi afişată de 8 ori

c) valoarea 1000 nu va fi afişată, deoarece pentru `nivel = 1000` condiţia `nivel < 1000` este falsă

d) valoarea 3 va fi afişată de 8 ori, deoarece la nivelurile 0, 1 şi 2 numărul firelor se dublează

8. Considerăm următorul program Java:

```
class Text {
    private String text;
    public Text(String text) { this.text = text; }
    public void schimba(String text) { this.text = text; }
    public void schimba(Text text) { text = new Text("Mihai"); }
    public void copiaza(Text text) { this.text = text.text; }
    public String getText() { return text; }
}

public class Test {
    public static void main(String[] args) {
        Text a = new Text("Alex");
        Text b = new Text("Ion");
        a.schimba("Matei");
        b.schimba(new Text("Dan"));
        a.schimba(b);
        b.copiaza(new Text("Vlad"));
        System.out.println(a.getText() + " " + b.getText());
    }
}
```

După executarea programului, se va afişa:

a) `Matei Vlad` **b)** `Matei Ion` **c)** `Mihai Ion` **d)** `Alex Vlad`

9. Considerăm următorul cod Java:

```
class A {
    int intA;
    public A met1() { return new A(); }
    private void met2() {}
    public void met3() {}
    public int met4(int i) { return 5; }
}
```

```
class B extends A {
    int intB;
    public B met1() { return new B(); }
    public void met2() {}
    private void met3() {}
    public static int met4() { return 5; }
}
```

Care dintre următoarele variante referitoare la codul de mai sus sunt corecte?

a) metoda `met1()` din clasa `B` suprascrie metoda `met1()` din clasa `A`, deoarece are aceeaşi listă de parametri, iar tipul returnat `B` este compatibil covariant cu tipul `A`

b) metoda `met2()` din clasa `B` suprascrie metoda `met2()` din clasa `A`, deoarece ambele au acelaşi nume şi aceeaşi listă de parametri

c) metoda `met3()` din clasa `B` produce eroare de compilare, deoarece încearcă să reducă vizibilitatea unei metode `public` moştenite din clasa `A`

d) metoda `met4()` din clasa `B` produce eroare de compilare, deoarece o metodă statică nu poate suprascrie o metodă non-statică cu acelaşi nume din clasa `A`

10. Considerăm următorul program Java:

```
class Test {
    static int x = 10;
    static { x += 5; }
    { x += 3; }
    Test() { x += 2; }
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        System.out.println(x);
    }
}
```

Care dintre următoarele afirmaţii referitoare la o codul de mai sus sunt adevărate?

a) programul afişează valoarea 26

b) blocul static de iniţializare se execută o singură dată, înainte de apelarea metodei `main()`

c) blocul non-static de iniţializare se execută de fiecare dată când se creează un obiect, înaintea constructorului clasei

d) dacă data membră `x` ar fi declarată nestatică, atunci ar apărea o eroare la compilare

UNIVERSITATEA DIN BUCUREŞTI **STUDENT: _____**
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ **GRUPA: _____**
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 1

II. Se consideră complet definită o clasă `JucatoareWTA` cu datele membre `nume`, `pozitieWTA`, `puncteWTA`, `naționalitate` și `nrTurnee`. Clasa este utilizată pentru a memora informații despre jucătoarele din Women's Tennis Association (WTA). Datele membre `nume`, `naționalitate` sunt de tip `String`, iar `pozitieWTA`, `puncteWTA` și `nrTurnee` sunt de tip `int`. Clasa încapsulează constructori, metode de tip `set`/`get` pentru toate datele membre, precum și metodele `toString()`, `equals()` și `hashCode()`. Creați o lista care să conțină cel puțin 3 obiecte de tip `JucatoareWTA` și, folosind stream-uri bazate pe lista creată și lambda expresii, rezolvați următoarele cerințe:

a) afișați informații despre toate jucătoarele care au care participat la cel puțin 10 turnee, sortate alfabetic după nume

b) afișați naționalitățile distincte ale jucătoarelor

c) afișați topul primelor 3 jucătoare (în funcție de punctaje)

d) creați o listă care să conțină jucătoarele care au acumulat intre 3000 si 7000 de puncte

e) pentru fiecare naționalitate să se afișeze numărul jucătoarelor având naționalitatea respectivă

III. Se consideră complet definită clasa mutabilă `DateContact`, care permite memorarea datelor de contact ale unei persoane: număr de telefon, email, adresă, oraș, județ și cod poștal. Definiți o clasă imutabilă `Angajat` care să permită memorarea următoarelor informații despre un angajat: datele sale de contact (referință spre un obiect de tip `DateContact`), numărul de competențe profesionale (număr natural), salariul (număr real) și competențele sale (listă cu elemente de tip `StringBuilder`).

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 1

STUDENT: _____
GRUPA: _____

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 1

STUDENT: _____
GRUPA: _____

EXAMEN LA DISCIPLINA "PROGRAMARE AVANSATĂ PE OBIECTE ÎN JAVA" – SESIUNEA IUNIE 2026 –

NOTĂ:

- Datele de intrare se consideră corecte.
- Nu se vor trata excepţiile şi importurile.
- Punctaj: 5p. (10 x 0.5p.) + 2p. + 2p. + 1p. (din oficiu)

II. Pentru fiecare dintre cele 5 întrebări de mai jos, indicaţi variantele de răspuns pe care le consideraţi corecte:

1. Într-o aplicaţie de gestiune urbană considerăm clasele `Cartier`, `Casa`, `Camera` şi `Proprietar`. Specificaţiile funcţionale ale aplicaţiei sunt următoarele:

- `Casa` este compusă din mai multe obiecte de tip `Camera`. Dacă o casă este demolată, atunci toate camerele sale dispar instantaneu.
- Fiecare `Casa` se află într-un `Cartier`. Dacă un `Cartier` este desfiinţat sau reorganizat administrativ, casele nu dispar, ci sunt mutate în mod dinamic în administrarea altui cartier.
- Fiecare `Casa` are un `Proprietar`. Dacă o casă este vândută sau demolată, obiectul `Proprietar` nu trebuie afectat, deoarece acesta poate deţine şi alte proprietăţi în aplicaţie sau poate rămâne în baza de date ca utilizator fără nicio casă.

Care dintre următoarele variante este o abordare corectă de modelare a acestor specificaţii din punct de vedere al programării orientate pe obiecte?

a) se defineşte şi o clasă abstractă `Oraş` pe care o vor extinde direct toate cele 4 clase precizate anterior
b) clasa `Casa` extinde clasa `Cartier`, clasa `Camera` extinde clasa `Casa`, iar între clasele `Proprietar` şi `Casa` avem o relaţie de agregare
c) între clasele `Cartier` şi `Casa` avem o relaţie de compoziţie, între clasele `Casa` şi `Camera` avem o relaţie de agregare, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
d) între clasele `Casa` şi `Camera` avem o relaţie de compoziţie, între clasele `Cartier` şi `Casa` avem o relaţie de agregare, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare

2. Considerăm următorul program Java:

```
class A { public void print() { System.out.print("A "); } }
class B extends A { }
class C extends B { public void print() { System.out.print("B "); } }
public class Test {
    public static void main(String args[]) {
        A p = new B();
        B q = new B();
        A r = new C();
        p.print();
        q.print();
        r.print();
    }
}
```

După executarea programului, se va afişa:

a) `A A B` **b)** `B B B` **c)** `A B B` **d)** `A B A`

3. Ştiind faptul că şirul de caractere `s` conţine doar paranteze rotunde, deschise sau închise, ce se va afişa pe ecran după executarea secvenţei de mai jos?

```
int n = 0;
int p = s.indexOf("()");
while (p != -1) {
    n++;
    s = s.substring(p + 2);
    p = s.indexOf("()");
}
System.out.println(n);
```

a) numărul de paranteze rămase în şirul `s` după eliminarea tuturor perechilor corect închise
b) numărul de grupări de paranteze imbricate corect care conţin cel puţin o pereche de paranteze în interiorul altei perechi, de exemplu `(())`, `((()))` ş.a.m.d., din şirul `s`
c) numărul de paranteze din prima grupare consecutivă de perechi `()` din şirul `s`
d) numărul perechi `()` care apar în şirul iniţial `s`, indiferent dacă sunt consecutive sau nu

4. Considerăm următorul program Java:

```
public class Test {
    public static void main(String args[]) throws FileNotFoundException {
        Scanner fin = new Scanner(new File("numere.txt"));
        int s = 0;

        while (fin.hasNext()) {
            try {
                int x = fin.nextInt();
                s = s + x;
            }
            catch (InputMismatchException e) {
                System.out.println("Număr incorect!");
                fin.next();
            }
        }
        System.out.println(s);
    }
}
```

Dacă în fişierul text *numere.txt* există cel puţin două numere scrise incorect (de exemplu: 2a17, a217, abc, 2-17 etc.), atunci programul:

a) va afişa o singură dată mesajul `Număr incorect!` şi suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina normal
b) va afişa câte un mesaj `Număr incorect!` pentru fiecare număr din fişier scris incorect, apoi va afişa suma tuturor numerelor din fişier scrise corect, după care programul se va termina normal
c) va afişa doar mesajul `Număr incorect!`, după care programul se va termina forţat din cauza unei excepţii
d) va afişa suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina forţat din cauza unei excepţii

5. Fie următorul program Java:

```
class Automobil {
    private String marca;
    public Automobil(String marca) { this.marca = marca; }
    public int hashCode() { return marca.length() / 2; }
    public boolean equals(Object obj) {
        return marca.charAt(0) != ((Automobil)obj).marca.charAt(0); }
}

public class Test {
    public static void main(String[] args) {
        HashSet<Automobil> ma = new HashSet<>();
        ma.add(new Automobil("Dacia")); ma.add(new Automobil("Bugatti"));
        ma.add(new Automobil("Ford"));  ma.add(new Automobil("Buick"));
        ma.add(new Automobil("Fiat"));  ma.add(new Automobil("Delage"));
        System.out.println(ma.size());
    }
}
```

După executarea programului, se va afişa valoarea:
a) 4 **b)** 3 **c)** 2 **d)** 1

6. Care dintre următoarele afirmaţii referitoare la o clasă abstractă sunt false?
a) constructorul unei clase abstracte `C` poate fi apelat direct folosind expresia `C.C()`
b) o clasă abstractă nu poate avea constructori, deoarece nu poate fi instanţiată direct cu operatorul `new`
c) o clasă abstractă poate avea constructori, iar aceştia vor fi apelaţi la crearea obiectelor claselor derivate, implicit sau explicit folosind cuvântul cheie `super`
d) toţi constructorii unei clase abstracte trebuie să fie obligatoriu `private`, pentru a împiedica instanţierea directă a clasei

7. Considerăm următorul cod Java:

```
class Fir extends Thread {
    int nivel;
    public Fir(int n) { nivel = n; }
    public void run() {
        System.out.print(nivel + " ");
        if (nivel < 3) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
        if (nivel < 1000) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Fir fir = new Fir(0);
        fir.start();
    }
}
```

Presupunând că toate firele create în programul de mai sus reuşesc să se execute complet, care dintre următoarele afirmaţii sunt adevărate?

a) valoarea 3 va fi afişată de 8 ori, deoarece la nivelurile 0, 1 şi 2 numărul firelor se dublează

b) valorile afişate vor apărea obligatoriu în ordine crescătoare, deoarece fiecare fir cu nivelul `n` creează fire doar cu nivelul `n + 1`

c) valoarea 999 va fi afişată de 8 ori

d) valoarea 1000 nu va fi afişată, deoarece pentru `nivel = 1000` condiţia `nivel < 1000` este falsă

8. Considerăm următorul program Java:

```
class Test {
    static int x = 10;
    static { x += 5; }
    { x += 3; }
    Test() { x += 2; }
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        System.out.println(x);
    }
}
```

Care dintre următoarele afirmaţii referitoare la o codul de mai sus sunt adevărate?

a) blocul static de iniţializare se execută o singură dată, înainte de apelarea metodei `main()`

b) blocul non-static de iniţializare se execută de fiecare dată când se creează un obiect, înaintea constructorului clasei

c) programul afişează valoarea 26

d) dacă data membră `x` ar fi declarată nestatică, atunci ar apărea o eroare la compilare

9. Considerăm următorul cod Java:

```
class A {
    int intA;
    public A met1() { return new A(); }
    private void met2() {}
    public void met3() {}
    public int met4(int i) { return 5; }
}
```

```
class B extends A {
    int intB;
    public B met1() { return new B(); }
    public void met2() {}
    private void met3() {}
    public static int met4() { return 5; }
}
```

Care dintre următoarele variante referitoare la codul de mai sus sunt corecte?

a) metoda `met1()` din clasa `B` suprascrie metoda `met1()` din clasa `A`, deoarece are aceeaşi listă de parametri, iar tipul returnat `B` este compatibil covariant cu tipul `A`

b) metoda `met2()` din clasa `B` suprascrie metoda `met2()` din clasa `A`, deoarece ambele au acelaşi nume şi aceeaşi listă de parametri

c) metoda `met3()` din clasa `B` produce eroare de compilare, deoarece încearcă să reducă vizibilitatea unei metode `public` moştenite din clasa `A`

d) metoda `met4()` din clasa `B` produce eroare de compilare, deoarece o metodă statică nu poate suprascrie o metodă non-statică cu acelaşi nume din clasa `A`

10. Considerăm următorul program Java:

```
class Text {
    private String text;
    public Text(String text) { this.text = text; }
    public void schimba(String text) { this.text = text; }
    public void schimba(Text text) { text = new Text("Mihai"); }
    public void copiaza(Text text) { this.text = text.text; }
    public String getText() { return text; }
}

public class Test {
    public static void main(String[] args) {
        Text a = new Text("Alex");
        Text b = new Text("Ion");
        a.schimba("Matei");
        b.schimba(new Text("Dan"));
        a.schimba(b);
        b.copiaza(new Text("Vlad"));
        System.out.println(a.getText() + " " + b.getText());
    }
}
```

După executarea programului, se va afişa:

a) `Matei Ion` **b)** `Matei Vlad` **c)** `Mihai Ion` **d)** `Alex Vlad`

II. Se consideră complet definită o clasă `JucatoareWTA` cu datele membre `nume`, `pozitieWTA`, `puncteWTA`, `naționalitate` și `nrTurnee`. Clasa este utilizată pentru a memora informații despre jucătoarele din Women's Tennis Association (WTA). Datele membre `nume`, `naționalitate` sunt de tip `String`, iar `pozitieWTA`, `puncteWTA` și `nrTurnee` sunt de tip `int`. Clasa încapsulează constructori, metode de tip `set`/`get` pentru toate datele membre, precum și metodele `toString()`, `equals()` și `hashCode()`. Creați o lista care să conțină cel puțin 3 obiecte de tip `JucatoareWTA` și, folosind stream-uri bazate pe lista creată și lambda expresii, rezolvați următoarele cerințe:

a) afișați informații despre toate jucătoarele care au care participat la cel mult 10 turnee

b) afișați naționalitățile distincte ale jucătoarelor, sortate alfabetic

c) afișați topul primelor 3 jucătoare (în funcție de punctaje)

d) creați o listă care să conțină jucătoarele care nu au acumulat intre 3000 si 7000 de puncte

e) pentru fiecare punctaj să se afișeze numele jucătoarelor având punctajul respectiv

III. Se consideră definită complet clasa imutabilă `Adresa` care permite memorarea unei adrese din România: stradă, număr, bloc, scară, etaj, apartament, judeţ, localitate şi cod poştal. Definiţi o clasă imutabilă `Imobil` care să permită memorarea următoarelor informaţii despre un imobil: *adresă* (referinţă spre un obiect de tip `Adresa`), *număr de locatari* (număr natural) şi *locatari* (listă cu elemente de tip `StringBuffer` care conţine numele locatarilor).

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 1

STUDENT: _____
GRUPA: _____